

CSE 144 Final Project Report

Transfer Learning Challenge

Connor Juricich (CruzID: `cjuricic`) John Gmuender (CruzID: `jgmuende`)

Spring 2026

1 Introduction

1.1 Problem Goal and Setting

The goal of this project is to classify images into one of 100 categories using a small labeled dataset. Each category contains approximately 10 training images, making the problem challenging due to the limited amount of training data available for each class. The final objective is to predict labels for the unlabeled test set and submit predictions to the Kaggle competition.

1.2 Why Transfer Learning is Appropriate

Transfer learning is well suited for this task because the dataset is relatively small. Training a large vision model from scratch would likely lead to severe overfitting. Instead, we leverage a pretrained vision transformer that has already learned useful visual features from large-scale image datasets and adapt it to our classification task.

1.3 Summary of Approach

Our final approach uses SigLIP-2 Vision Transformer (`google/siglip2-base-patch16-224`) as a frozen feature extractor. Images are converted into embedding vectors using the pretrained backbone, and a lightweight multilayer perceptron (MLP) classifier is trained on top of those embeddings.

This approach allows us to benefit from powerful pretrained visual representations while only training a small number of parameters on the project dataset.

2 Dataset

2.1 Dataset Size

The dataset consists of:

- 100 image classes
- Approximately 10 training images per class
- Approximately 1000 labeled training images total
- 1036 unlabeled test images

For model development, the training data was split into:

- 80% Training
- 20% Validation

2.2 Directory Structure and Label Mapping

The training directory contains folders labeled from 0 through 99. Each folder corresponds directly to its class label. During dataset loading, folder names are sorted and mapped to class indices, ensuring that label 0 corresponds to folder 0, label 1 corresponds to folder 1, and so on.

2.3 Preprocessing

Images are processed using the SigLIP-2 image processor provided by HuggingFace Transformers. The processor automatically applies the resizing and normalization expected by the pretrained model.

3 Implementation

3.1 Model Architecture

The pretrained backbone used in this project is:

- SigLIP-2 Vision Transformer
- `google/siglip2-base-patch16-224`

The backbone remains frozen throughout training and is used only to generate image embeddings.

A lightweight classifier head is trained on top of the extracted embeddings:

- Linear layer: embedding dimension $\rightarrow$ 512
- ReLU activation
- Dropout ($p = 0.2$)
- Linear layer: 512 $\rightarrow$ 100 classes

3.2 Fine-Tuning Strategy

Rather than fine-tuning the entire backbone, all SigLIP-2 parameters were frozen. Only the classifier head was trained. This significantly reduced training time and helped prevent overfitting on the small dataset.

3.3 Training Configuration

Hyperparameter	Value
Backbone	SigLIP-2 Base
Epochs	35
Batch Size	32
Optimizer	AdamW
Learning Rate	1×10^{-4}
Weight Decay	0.05
Dropout	0.2
Loss Function	Cross Entropy Loss
Random Seed	42

Table 1: Training Hyperparameters

3.4 Hardware and Software

Training and inference were performed using PyTorch and HuggingFace Transformers on a CUDA-enabled GPU environment.

4 Experiments

4.1 Validation Method

An 80/20 train-validation split was used to evaluate model performance during training. Validation accuracy was measured after each epoch.

4.2 Training Procedure

For each batch:

1. Images are processed using the SigLIP-2 image processor.
2. Embeddings are extracted from the frozen backbone.
3. Embeddings are passed through the classifier head.
4. Cross entropy loss is computed.
5. The classifier head parameters are updated using AdamW.

The best validation checkpoint is saved during training.

5 Results

5.1 Training Performance

Validation accuracy improved steadily throughout training and converged after approximately 30 epochs.

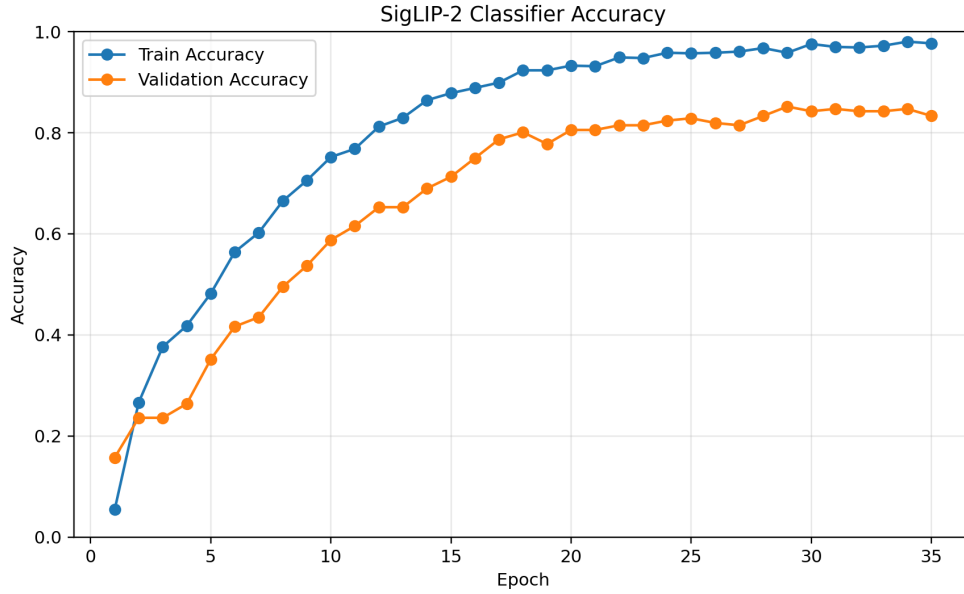


Figure 1: Validation Accuracy During Training

Training loss also decreased consistently throughout optimization, indicating stable learning behavior.

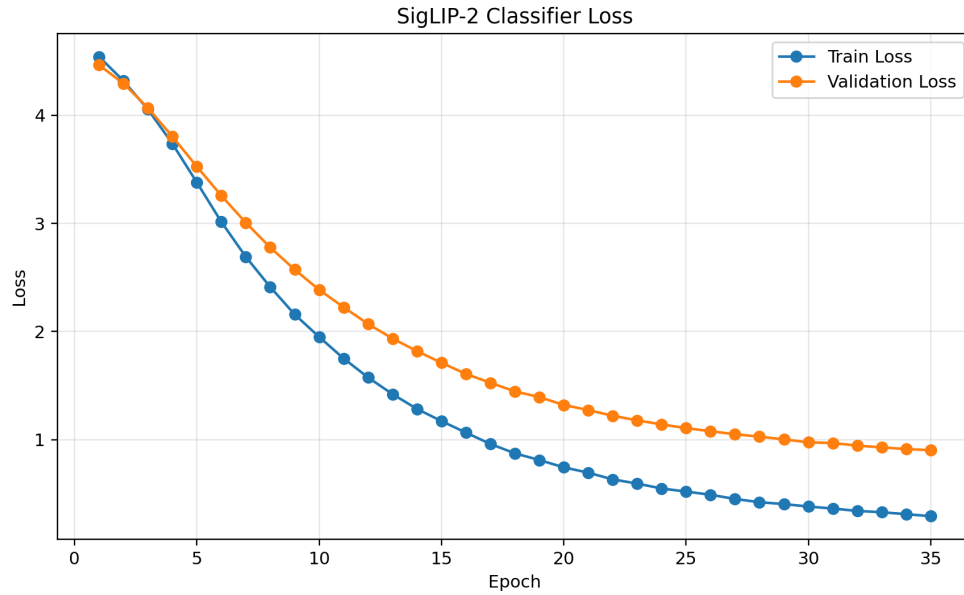


Figure 2: Training Loss During Training

5.2 Kaggle Submission

Predictions were generated using the best validation checkpoint and submitted to the Kaggle competition leaderboard.



Figure 3: Kaggle Public Leaderboard

Public Leaderboard Score: 0.90909

6 Discussion

6.1 What Worked Best

The strongest contributor to performance was the use of the pretrained SigLIP-2 backbone. Because the model was trained on a large-scale image dataset, it was able to produce high-quality image representations even for classes with very few examples.

Training only the classifier head provided a good balance between computational efficiency and performance.

6.2 Limitations

The primary limitation of this project is the small number of training images per class. With only approximately 10 images per category, there is limited opportunity for the model to learn dataset-specific features.

Additionally, the backbone was frozen and therefore could not fully adapt to the target dataset.

6.3 Future Improvements

Potential future improvements include:

- Fine-tuning the later layers of SigLIP-2
- Additional data augmentation techniques
- Hyperparameter optimization
- Cross-validation for more reliable performance estimates
- Training multiple classifier heads and ensembling predictions

7 Reproducibility

Experiments were conducted with a fixed random seed of 42 and a fixed 80/20 train-validation split. The complete source code, trained model weights, and instructions for training and inference are provided in the project repository.

Package	Version
PyTorch	2.11.0+cu128
Transformers	5.10.1
NumPy	2.0.2
Pillow	11.3.0
tqdm	4.67.3

Table 2: Software environment used for training and inference.

Training:

```
python train.py
```

Inference:

```
python inference.py
```

The inference script loads `siglip2_classifier.pt` and generates `submission.csv`.

8 Team Contributions

Connor Jurich

- Designed and implemented the training pipeline
- Implemented SigLIP-2 feature extraction
- Developed the classifier head
- Trained and evaluated the final model
- Generated Kaggle submissions

John Gmuender

- Assisted with model evaluation and result analysis
- Prepared presentation materials
- Assisted with report writing and documentation
- Reviewed experimental results

9 References

1. HuggingFace Transformers Documentation
<https://huggingface.co/docs/transformers>
2. SigLIP-2 Model Documentation
<https://huggingface.co/google/siglip2-base-patch16-224>
3. PyTorch Documentation
<https://pytorch.org/docs/stable>